

SE Lab 4: VibeCoding — AI Chat Interaction Record

Project: Word Scramble Arena (Pygame)

AI tools used: ChatGPT (bug-fix guidance) and GitHub Copilot in VS Code (feature implementation).

Evidence note: The Copilot sections below are readable excerpts taken from the actual exported VS Code chat session. System/tool internals, lengthy code patches, and repeated status lines were omitted. The ChatGPT section is an excerpt from the actual project guidance conversation, not a Copilot export. This is a documented extract, not a full verbatim transcript.

Task 1 — Fix correct-answer validation | ChatGPT

User supplied code (relevant line)

```
is_correct = (guess == self.scrambled_word)
```

ChatGPT response (excerpt)

We found the exact bug in your Word Scramble Game! The wrong code checks whether your answer matches the scrambled letters, not the actual word. Replace it with: `is_correct = (guess == self.secret_word)`. Save the file and run `python main.py` to test.

Observed follow-up

The user confirmed the game worked after this correction. The change was committed separately as Task 1.

Task 2 — Hint button | GitHub Copilot (VS Code)

User prompt (exact exported text)

Add a Hint button to my existing Pygame Word Scramble game. Each click should reveal one more letter of the correct word in order. Deduct 1 score point per hint, but never let the score go below zero. Reset the hints when a new word appears. Keep all existing game functionality working. Modify only the necessary files.

Copilot responses (excerpts from exported chat)

I'll trace the round lifecycle and rendering in the existing engine, then make the smallest focused change and run a quick syntax or behavior check. I'll also check the repo's local instructions before editing.

The engine owns both round changes and mouse handling, so the feature fits entirely in

. I'll track how many leading letters are exposed, reset that count in

, and only charge for a click that reveals a new letter; this keeps repeated clicks after the whole word is shown from charging for a no-op.

The focused runtime check passes, including the score floor and round reset. I spotted one tiny presentation issue in the masked string: the first revealed letter runs directly into the next placeholder, so I'm spacing the displayed characters consistently and rerunning the same check.

Added a Hint button and ordered letter reveals in

. Each new reveal costs 1 point, the score stays at zero or above, and hints reset when a new word appears. Existing submit behavior is unchanged.

Verified with a focused runtime check covering hint progression, score clamping, completed reveals, and round reset.

Test output recorded in session

```
t==0; pygame.quit(); print('Hint behavior checks passed')"
```

Hint behavior checks passed

Task 3 — 30-second countdown timer | GitHub Copilot (VS Code)

User prompt (exact exported text)

Add a 30-second countdown timer to my existing Pygame Word Scramble game. Display the remaining seconds clearly on screen. Start the timer whenever a new word appears. When the timer reaches zero, automatically move to a new scrambled word and reset the timer. Reset the timer after a correct answer too. Use `pygame.time.get_ticks()` without blocking the game loop. Preserve the existing Hint button, score system, and Submit functionality. Modify only necessary files and test the timer.

Copilot responses (excerpts from exported chat)

I'll use the existing round transition as the timer's reset point and

for the non-blocking expiration check. First I'm confirming the current engine structure; then I'll add the countdown display and exercise both timeout and correct-answer resets with a controlled clock.

The round lifecycle confirms the simplest control path:

already runs at startup and after correct answers. I'll anchor the 30,000 ms start time there, advance rounds from the existing per-frame

The first test invocation returned to the PowerShell prompt without running the test body, so it didn't give us a result. I'm rerunning the same controlled-clock checks as a single Python command to verify expiration and both round-reset paths.

, so the loop stays non-blocking. Timeout and correct answers both start a fresh timer; the Hint button and score behavior remain intact.

Verified countdown rendering, the 30-second timeout, automatic round change, and correct-answer reset using a mocked clock.

Test output recorded in session

```
answer reset checks passed')"
```

Timer timeout, countdown rendering, and correct-answer reset checks passed

Task 4 — Interactive clickable letter tiles | GitHub Copilot (VS Code)

User prompt (exact exported text)

Add interactive clickable letter tiles to my existing Pygame Word Scramble game. Display each scrambled letter inside a separate clickable tile. When a player clicks a tile, append its letter to the answer input and mark the tile as selected so it cannot be used twice. Provide a way to undo or clear selected letters. Correctly handle repeated letters, such as the two Zs in PUZZLE. Keep keyboard input working. Reset tile selections for every new word. Preserve the Hint button, 30-second countdown timer, score system, and Submit functionality. Modify only necessary files and test the feature.

Copilot responses (excerpts from exported chat)

I'll keep this inside the game engine: tile hitboxes can follow the scrambled word, and per-character source tracking will let Undo release the correct tile even when keyboard input is mixed in. I'm checking the current input and click flow once, then I'll add the tiles plus Undo/Clear controls and test repeated letters and keyboard coexistence.

The current flow supports this without touching

: I can observe its normal keyboard edits and keep tile/source state synchronized in the engine. I'm adding index-based selection (so repeated letters remain distinct), resetting it on wrong guesses and new rounds, and placing Undo/Clear below the answer row.

The tile state now tracks positions rather than letter values, so both `Z` tiles in `PUZZLE` are independently selectable. I've also made keyboard additions/backspaces update that state, which lets Undo and Clear work predictably alongside typed letters; next I'm exercising those interactions and a new-round reset.

The first interaction check exposed a real keyboard-focus issue:

deactivates itself on tile clicks, so subsequent typing would be ignored. I'm keeping the input focused when a game control or tile is clicked, while retaining the normal click-to-focus behavior for the answer box; then I'll rerun the mixed-input test.

The mixed interaction check now passes, including separate `Z` selection, duplicate-click blocking, Undo, Clear, typing after tile clicks, and resetting selections for a new round. I'm doing one final integration check on the preserved Hint, Submit/score, and countdown behavior.

Added clickable letter tiles, with each position tracked separately so repeated letters such as the two `Z`s in `PUZZLE` work independently. Added Undo and Clear controls, kept keyboard typing active after tile clicks, and reset tile selections for each new word. Changes are limited to

.

Verified repeated-letter selection, duplicate-click blocking, Undo/Clear, mixed keyboard input, round resets, Hint deductions, Submit scoring, timer timeout/reset, and rendering.

Test output recorded in session

```
answer_sources; e.render(pygame.Surface((700,500))); pygame.quit(); print('Tile selection, duplicate letters, undo, clear, keyboard sync, round reset, and rendering checks passed')
```

```
AssertionError
```

```
Tile selection, duplicate letters, undo, clear, keyboard sync, round reset, and rendering checks passed
```

```
Hint, submit score, timer reset/timeout, and full render checks passed
```

Git version history

Separate commits in the assigned 62_scramble repository: Task 1 (41258b4), Task 2 (e1f4eb7), Task 3 (4fa8d7e), Task 4 (49a6ef3). The Lab-4 deliverables were also uploaded to the personal SE Lab repository (4ae149d).